

Tattava Copilot — Technology Architecture & Engineering Specification

Version: 1.0

Date: September 2026

Product: Tattava Copilot — V1 Story Development Copilot / Narrative Intelligence

Audience: Product, Engineering, AI/ML, Data, QA, Security, DevOps/SRE, Leadership

0. Document Status & Source Discipline

This specification synthesizes the attached **BRD, Product Discovery Artifact, AI-Native Product Strategy, V1 Product Definition & Planning, and Unified AI-Native Product Specification**, with the engineering architecture material already derived from those sources.

The source hierarchy is:

1. **BRD** — business intent, V1 capability breadth and non-functional requirements.
2. **Product Discovery** — validated vs. unvalidated assumptions and MVP hypotheses.
3. **AI-Native Product Strategy** — AI-native principles, strategic architecture and Golden Workflow.
4. **V1 Product Definition & Planning** — detailed product requirements, entities, user journeys and approval behavior.
5. **Unified AI-Native Product Specification** — consolidated AI/agent/evaluation operating model.
6. **Engineering specification material** — implementation-level architecture where already derived from the above.

The Discovery artifact explicitly states that the complete V1 capability set is **not yet empirically validated**; its current MVP hypothesis is Persistent Project Intelligence + Research + Context-Aware Story Development + Canon/Continuity + Human Approval.

Status labels

Label	Meaning
Source Requirement	Explicitly stated in source material
Derived Requirement	Technically necessary consequence of a source requirement
Recommendation	Proposed implementation choice
TBD / Open Decision	Source material does not determine the answer
Future	Explicitly outside V1

1. Executive Summary

Tattava V1 should be implemented as an **AI-native Project Intelligence system**, not as a collection of independent AI agents or a generic chatbot.

The central technical object is the **Project Intelligence Layer / Story Brain**. It maintains structured, persistent project state covering:

- project
- premise/story
- characters
- relationships
- research/evidence
- creative decisions
- canon
- narrative structure
- artifacts
- versions
- approvals
- dependencies

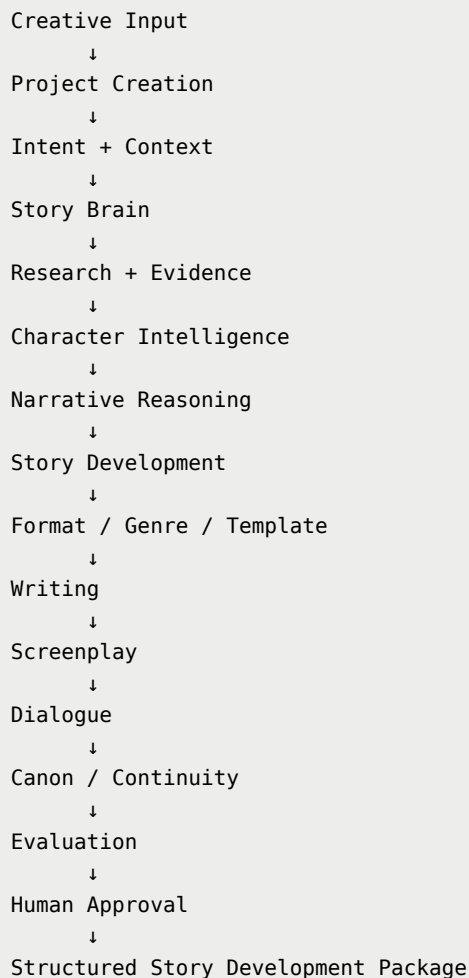
The product then places an orchestration and AI layer above this state.

The core architectural principle is:

AI may reason, retrieve, generate, critique and recommend; deterministic application services govern identity, authorization, project isolation, state mutation, versioning and approvals.

This follows the source principle that AI performs cognitive work while humans retain creative authority.

The target V1 flow is:



The source strategy describes this as the Golden Workflow, while Discovery cautions that not every capability should automatically become MVP scope.

2. Source & Requirement Synthesis

2.1 Product requirements that directly affect technology

The technology architecture must support five fundamental product problems:

Product problem	Technology implication
Fragmented project context	Persistent structured Project Intelligence
Repetitive development work	Agent/workflow orchestration
Research traceability	Evidence + provenance model
Continuity risk	Canon + relationship/dependency model
Creative handoff friction	Shared project state consumed by downstream capabilities

The Strategy explicitly describes Tattava's distinction as a system that understands project information and reasons over it rather than merely storing it.

2.2 Core technical principles

P1 — Project over prompt

Every meaningful AI request must be associated with a project.

P2 — Context over generation

The system retrieves relevant project state rather than dumping the entire project into an LLM.

P3 — Evidence before confidence

Research should follow:

```
Source
↓
Evidence
↓
Finding
↓
Project Relevance
↓
Decision
```

P4 — Human authority

```
AI proposes
  ↓
Human reviews
  ↓
Human edits / accepts / rejects
  ↓
System records decision
  ↓
Approved state becomes canonical
```

P5 — Traceability

Important project information must be traceable to:

- source
- version
- actor
- AI run
- decision
- approval
- downstream impact

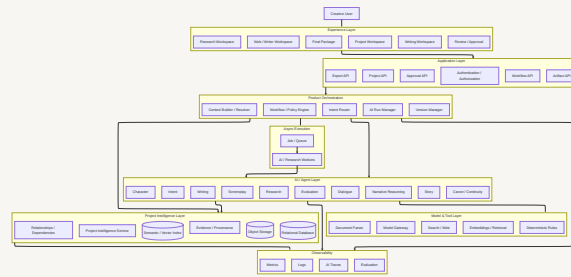
P6 — Model agnostic

The product's durable architecture should sit **above model providers**, not become coupled to one LLM.

These principles are explicitly established in the strategy and unified specification.

3. Architecture Blueprint

3.1 Target logical architecture



The engineering architecture already derived from the source material uses this same logical separation: Web/Writer UI → API/AuthZ → Workflow/Policy → Agent Orchestrator → AI agents/tools/rules → Project Intelligence → persistence → observability.

3.2 Architectural layers

Layer 1 — Experience

Responsibilities:

- Project dashboard
- Story Brain
- Research
- Characters
- Story
- Writing
- Screenplay
- Canon
- Evaluation
- Versions
- Approval
- Final Package

Important: AI should appear inside these workflows rather than as a separate generic chatbot destination.

Layer 2 — Application/API

Responsibilities:

- Authentication
- Authorization
- Project access
- Commands
- Queries
- Artifact operations
- Approval operations
- Workflow execution
- Export

This layer is the public application contract and must never allow the frontend to bypass policy.

Layer 3 — Workflow / Policy Engine

This is one of the most important components.

Responsibilities:

- State transitions
- Approval gates
- Permission enforcement
- Version checks
- Job lifecycle
- Idempotency
- Retry policy
- Preconditions/postconditions
- Agent capability authorization

Critical rule:

LLM output must never directly mutate authoritative project state.

Layer 4 — Agent Orchestrator

Responsibilities:

- Select workflow
- Instantiate agent run
- Build context
- Call model gateway
- Invoke tools
- Validate output
- Persist candidate result
- Emit telemetry
- Handle retry/failure

The Unified Specification explicitly recommends that agent execution state remain temporary and non-authoritative, while the Story Brain remains the system of record.

Layer 5 — Project Intelligence Layer

This is the architectural core.

```
Project
├── Story
├── Characters
├── Relationships
├── World
├── Research
│   └── Evidence
├── Decisions
├── Canon
├── Artifacts
├── Versions
├── Approvals
└── Dependencies
```

The Story Brain must be a **structured state model**, not a giant prompt or chat transcript.

Layer 6 — Model / Tool Layer

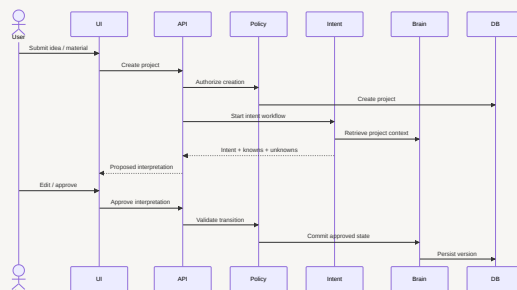
Abstract interfaces for:

- LLMs
- Search
- Web retrieval
- Document parsing
- Embeddings
- Semantic retrieval
- Deterministic rules

Specific providers are **TBD**.

4. Key Data Flows

4.1 Project creation



The source user journey explicitly requires initial interpretation to be editable and approved before becoming project intelligence.

4.2 Research



Research must distinguish:

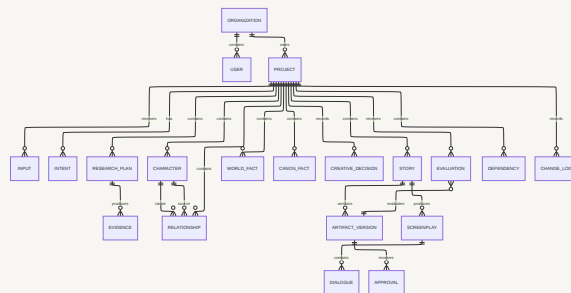
- sourced evidence

- AI inference
- unresolved uncertainty

and must not convert unsupported claims into authoritative project facts.

5. Data Model

5.1 Conceptual ER model



The engineering data model explicitly identifies Project, Input, Intent, ResearchPlan, Evidence, CreativeDecision, Character, Relationship, WorldFact, CanonFact, Story, Screenplay, Dialogue, Evaluation, Dependency, ArtifactVersion, Approval and ChangeLog.

5.2 Core entities

Organization

Purpose: tenancy boundary.

Suggested fields:

```
organization_id
name
status
created_at
updated_at
```

Status: Derived requirement / exact organization model TBD.

User

```
user_id
organization_id
identity_provider_id
name
email
status
created_at
updated_at
```

Project

```
project_id
organization_id
title
status
format
genre
owner_id
created_at
updated_at
```

The Project is the primary tenancy and intelligence boundary.

Input

Stores original project material.

```
input_id
project_id
input_type
object_storage_ref
raw_metadata
parsed_content
created_by
created_at
```

Examples:

- idea
- logline

- concept
- script/reference
- character ideas
- themes
- creative brief

Original uploaded files should remain immutable references; parsed representations are derived data.

Intent

```
intent_id
project_id
intent
knowns
unknowns
assumptions
status
source_run_id
created_at
```

Important distinction:

```
Known
Unknown
Assumption
Approved Fact
```

These should never be collapsed into one generic context field.

ResearchPlan

```
research_plan_id
project_id
questions
scope
status
created_by
created_at
```

Evidence

```
evidence_id
project_id
research_plan_id
source_url/reference
source_type
publisher
author
published_at
retrieved_at
claim
supporting_excerpt
relevance
confidence
validation_status
```

CreativeDecision

```
decision_id
project_id
statement
rationale
evidence_refs[]
status
approver_id
approved_at
```

Critical distinction:

Evidence answers:

| "What does the source support?"

CreativeDecision answers:

| "What did the creative team decide?"

They must not be conflated.

Character

```
character_id
project_id
```

```
name
profile
motivation
voice
status
current_version_id
```

Relationship

```
relationship_id
project_id
from_entity_id
to_entity_id
relationship_type
attributes
status
```

CanonFact

```
canon_fact_id
project_id
category
statement
source_ref
status
version_id
```

Canon is authoritative project truth.

Rule:

| Canon is never silently overwritten.

Story

```
story_id
project_id
premise
logline
synopsis
themes
conflict
```

```
beats
status
```

Story content is versioned through `ArtifactVersion`.

ArtifactVersion

```
artifact_version_id
artifact_type
artifact_id
version_number
parent_version_id
status
created_by
reason
created_at
```

Possible states:

```
AI_GENERATED
CANDIDATE
HUMAN_EDITED
HUMAN_APPROVED
CANONICAL
SUPERSEDED
REJECTED
STALE
```

The state model is explicitly defined in the Unified Specification.

Approval

```
approval_id
artifact_version_id
approver_id
decision
reason
timestamp
```

Dependency

```
dependency_id
project_id
upstream_entity
downstream_entity
dependency_type
strength
status
```

Examples:

```
Character → Story Beat
Story Beat → Scene
Scene → Dialogue
Canon Fact → Scene
Character → Screenplay
```

Evaluation

```
evaluation_id
project_id
artifact_version_id
rubric_version
findings
status
created_at
```

ChangeLog

```
change_log_id
project_id
entity_type
entity_id
actor_type
actor_id
action
before_ref
after_ref
run_id
timestamp
```

This provides traceability without exposing hidden model reasoning.

6. AI-Specific Data Model

AI execution should have separate operational entities.

AI Run

```
run_id
project_id
workflow_id
agent_id
agent_version
model_provider
model_id
model_version
prompt_version
input_refs[]
context_refs[]
tool_calls[]
output_ref
validation_status
status
latency_ms
token_usage
estimated_cost
error_code
created_at
completed_at
```

Prompt Version

```
prompt_id
agent_id
version
system_policy_ref
workflow_policy_ref
template
output_schema_ref
created_at
status
```

Retrieval Record

```
retrieval_id
run_id
```

```
project_id
query
entity_refs[]
document_refs[]
ranking_metadata
created_at
```

Feedback

```
feedback_id
project_id
artifact_version_id
run_id
user_id
feedback_type
feedback_value
comment
created_at
```

These structures are **derived engineering requirements**, because the Unified Specification requires AI runs, context, model, tools, outcomes, latency and cost to be observable.

7. Data Authority Model

The system should enforce the following hierarchy:

```
ORIGINAL SOURCE
↓
DERIVED / PARSED DATA
↓
AI CANDIDATE
↓
HUMAN EDITED
↓
HUMAN APPROVED
↓
CANONICAL PROJECT STATE
```

Never:

LLM output → Canon

Instead:

```
LLM output
↓
validation
↓
candidate
↓
human approval
↓
new version
↓
canonical state
```

This is central to preventing AI-generated errors from contaminating future context.

8. API / Integration Specification

8.1 API architecture

The source engineering specification defines example `/v1` endpoints but explicitly says final naming conventions are not locked.

Project APIs

```
POST /v1/projects
GET /v1/projects/{project_id}
GET /v1/projects/{project_id}/intelligence
```

AI workflow APIs

```
POST /v1/projects/{project_id}/agent-runs
GET /v1/agent-runs/{run_id}
```

Artifact APIs

```
GET /v1/artifacts/{artifact_id}/versions
POST /v1/artifacts/{artifact_id}/approve
POST /v1/artifacts/{artifact_id}/reject
POST /v1/artifacts/{artifact_id}/restore
```

Dependency

```
GET /v1/projects/{project_id}/impact
```

Export

```
POST /v1/projects/{project_id}/export
```

8.2 Example: Create Project

```
POST /v1/projects
Authorization: Bearer <token>
Idempotency-Key: <key>
Content-Type: application/json
```

```
{
  "title": "Project Rajyam",
  "format": "series",
  "genre": "historical drama",
  "input": {
    "type": "creative_brief",
    "content_ref": "...",
  }
}
```

Response:

```
{
  "project_id": "proj_123",
}
```

```
"status": "INITIALIZING"
}
```

8.3 Start AI workflow

POST /v1/projects/{project_id}/agent-runs

```
{
  "workflow_id": "story_development",
  "input": {
    "instruction": "Explore three narrative directions"
  },
  "context_refs": [
    "story:current",
    "characters:approved",
    "research:approved"
  ]
}
```

Response:

```
{
  "run_id": "run_123",
  "status": "QUEUED"
}
```

8.4 API rules

All mutation endpoints must:

- authenticate
- authorize against project membership
- validate current version
- validate workflow state

- enforce approval rules
- support idempotency where duplicate execution is dangerous
- return stable IDs
- return explicit version references
- use machine-readable error codes

Long-running AI operations should return a job/run identifier rather than blocking an HTTP request indefinitely.

These rules are already specified in the engineering architecture.

9. Authentication & Authorization

9.1 Authentication

TBD / Architecture Decision

Source material says an organizational identity provider should be used if available; otherwise an approved authentication service.

Do not lock a provider until organizational identity requirements are known.

9.2 Authorization

Minimum conceptual model:

```
Organization
  ↓
Project
  ↓
Project Membership
  ↓
Role
  ↓
Capability Permission
```

Potential permissions:

```
VIEW
EDIT
```

GENERATE
APPROVE
EXPORT
ADMIN

The exact role/permission matrix remains a Product decision. The V1 product specification identifies View/Edit/Generate/Approve but leaves parts of approval authority unresolved.

10. External Integrations

V1-relevant integration classes

1. LLM providers

Purpose:

- reasoning
- generation
- extraction
- evaluation

Provider:

TBD

Architecture:

```
Agent
  ↓
Model Gateway
  ↓
Provider Adapter
  ↓
LLM Provider
```

2. Search / Web

Purpose:

- project-specific research

- evidence retrieval

Requirements:

- allowlisted tools
 - controlled domains
 - minimum necessary context
 - provenance capture
 - source validation
 - audit logging
-

3. Document Parser

Purpose:

- ingest scripts
- briefs
- reference documents

Uploaded documents must be treated as **untrusted data**, not instructions.

4. Object Storage

Purpose:

- uploaded documents
- raw files
- exports
- potentially generated artifacts

Exact provider: **TBD**

5. Vector / Semantic Retrieval

Purpose:

- retrieve relevant project context
- semantic search across project material

Vector retrieval must **never replace authorization**.

The Unified Specification explicitly states that semantic similarity cannot be treated as an authorization mechanism.

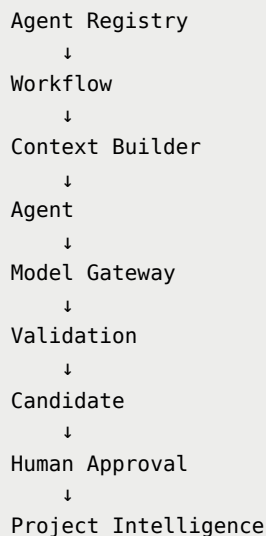
Future integration — DVS Production Suite

The BRD identifies DVS Production Suite integration as a future Layer 2 capability, not a V1 integration. Therefore it should influence the data contract but should **not block V1 implementation**.

11. AI / Agent Architecture

11.1 Agent model

Agents should be treated as **logical capabilities**, not necessarily independent deployable microservices.



The source architecture recommends a modular application plus worker subsystem rather than prematurely creating a microservice for every agent.

11.2 Agent contract

Every agent should define:

Field	Required
Agent ID	Yes
Agent version	Yes
Purpose	Yes
Trigger	Yes
Input schema	Yes
Context requirements	Yes
Retrieval strategy	Yes
Tools	Yes
Tool permissions	Yes
Model policy	Yes
Output schema	Yes
Validation	Yes
Allowed writes	Yes
Approval requirement	Yes
Failure behavior	Yes
Retry policy	Yes
Evaluation criteria	Yes
Telemetry	Yes

11.3 Agent catalogue

Agent	Primary responsibility	State mutation
Intent	Understand goal + unknowns	Candidate intent
Exploration	Generate directions	Candidate
Research	Find evidence	Evidence candidates
Character	Structure character intelligence	Candidate character state
Narrative Reasoning	Analyze logic	Diagnostics
Story	Develop narrative	Candidate story
Format/Genre	Advise/configure format	Configuration
Template	Apply deterministic structure	Configuration

Agent	Primary responsibility	State mutation
Writing	Draft/rewrite	Candidate artifact
Screenplay	Transform approved story	Candidate screenplay
Dialogue	Generate contextual dialogue	Candidate dialogue
Story Doctor	Diagnose issues	Diagnostics
Canon/Continuity	Detect contradictions	Findings/resolution
Evaluation	Evaluate readiness	Evaluation report
Impact	Identify downstream effects	Impact report

The Unified Specification defines these boundaries and explicitly prevents agents from silently mutating canonical state.

12. Context Resolution / RAG Architecture

12.1 Critical architecture rule

Do not send the complete Story Brain to every model call.

Instead:

```

User Request
  ↓
Identify capability
  ↓
Authorize project
  ↓
Identify required entities
  ↓
Retrieve canonical state
  ↓
Retrieve relevant approved artifacts
  ↓
Retrieve recent changes
  ↓
Retrieve dependencies
  ↓
Apply project/version filters
  ↓
Build minimum context package
  ↓
Agent

```

This is explicitly defined as a derived requirement in the Unified Specification.

12.2 Retrieval hierarchy

Recommended retrieval priority:

1. Current approved artifact
2. Relevant canon
3. Relevant characters
4. Relevant relationships
5. Relevant decisions
6. Relevant evidence
7. Relevant historical versions
8. Semantic search results

Authorization filtering happens **before or alongside retrieval**, not after an LLM receives the data.

12.3 Retrieval metadata

Every retrieved context element should carry:

```
entity_id
entity_type
project_id
version_id
status
authority_level
source_ref
relevance_reason
retrieval_timestamp
```

13. Engineering Build Specification

13.1 Recommended V1 deployment model

Recommendation

Use:

```
Managed Web App
  ↓
Modular Application/API
  ↓
Workflow / Policy Modules
  ↓
Async Worker Subsystem
  ↓
Managed Relational DB
  ↓
Object Storage
  ↓
Optional Semantic Index
```

Avoid prematurely splitting every logical agent into a separate microservice.

The engineering architecture recommends:

- managed PostgreSQL
- managed object storage
- asynchronous workers
- managed queue where appropriate
- optional Redis only when justified
- managed secrets
- automated CI/CD
- dev/staging/pilot/production separation.

13.2 Suggested repository structure

Recommendation

```
tattava/
|
├─ apps/
|   ├─ web/
|   └─ api/
|
```

```

├── packages/
│   ├── domain/
│   ├── schemas/
│   ├── auth/
│   ├── ai/
│   ├── retrieval/
│   ├── prompts/
│   ├── evaluation/
│   └── observability/
├── workers/
│   ├── research/
│   ├── generation/
│   ├── evaluation/
│   └── export/
├── infrastructure/
├── migrations/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   ├── security/
│   └── ai/
└── docs/

```

This is a **Recommendation**, not a source requirement.

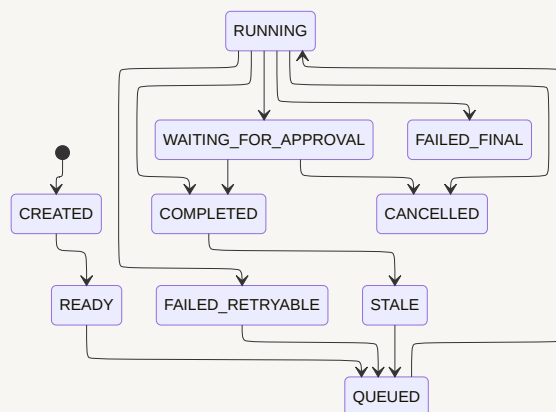
14. Technology Responsibility Matrix

Component	Responsibility	Technology status
Web UI	Creative workspace	Next.js/React recommended; exact stack TBD
API	Application contract	TypeScript/Python TBD
Workflow Engine	State + policies	Application module
Agent Orchestrator	AI execution	Application/worker module
Model Gateway	Provider abstraction	Required
Relational DB	Authoritative state	Managed PostgreSQL recommended
Object Storage	Files/exports	Managed storage recommended
Vector Index	Semantic retrieval	Optional / justified by retrieval need
Queue	Async work	Managed queue or existing org standard

Component	Responsibility	Technology status
Cache	Performance	Optional
Secrets	Credentials	Managed secret store
Observability	Logs/traces/metrics	Existing org standard preferred
IaC	Infrastructure	Existing org standard

The source engineering architecture identifies Next.js/React as a recommendation for the client and PostgreSQL/object/vector storage as the persistence direction, while leaving several physical technology choices open.

15. Workflow State Machine



The Unified Specification defines essentially this job lifecycle and requires asynchronous, resumable and idempotent execution.

16. Versioning Architecture

Version rules

Never mutate approved state directly.

Instead:

Approved v1
↓

```
User asks regenerate
↓
Candidate v2
↓
Human edit
↓
Human approval
↓
Approved v2
```

Old versions remain available.

Stale dependency model

```
Character v1
↓
Story v4
↓
Scene v2
↓
Dialogue v3
```

If Character changes:

```
Character v2
↓
Story v4 = STALE
Scene v2 = STALE
Dialogue v3 = STALE
```

The system should surface the impact rather than silently regenerate downstream content.

The strategy explicitly identifies dependency intelligence as a future evolution of the Story Brain.

17. Security, Privacy & Compliance

17.1 Mandatory controls

Project isolation

Every project query must include:

```
organization_id
+
project_id
+
authorized principal
```

Authorization

Enforced server-side.

Never trust:

- frontend permission state
- prompt instructions
- LLM decisions
- vector search boundaries

Encryption

Source engineering material requires:

- TLS in transit
- managed encryption at rest

Exact algorithms/key management: **TBD / organizational standard.**

Secrets

Never store:

- API keys
- provider credentials
- database credentials

inside:

- source code
- prompts
- project records

- user-visible traces.

Audit

Material state changes record:

```
actor
actor type
target
action
timestamp
reason
version
AI run
```

18. AI-Specific Threat Model

Threat	Required control
Prompt injection	Treat documents/web as untrusted data
Indirect injection	Separate data from system instructions
Data exfiltration	Minimize external context
Tool abuse	Tool allowlist + domain restrictions
Cross-project leakage	Project-scoped queries
Hallucinated research	Evidence + provenance validation
Unauthorized state mutation	Server-side workflow gate
Provider misuse	Provider data-policy controls
Cost abuse	Usage/cost telemetry + future budgets

These controls are explicitly reflected in the engineering security model.

19. AI Engineering Specification

AI-WF-01 — Intent

```
Trigger:
  User submits project input
```

Context:
Input + project metadata + recent relevant state

Retrieval:
Project scoped

External tools:
None by default

Output:
Structured intent JSON

Validation:
Schema validation
Known/unknown validation

Failure:
Ask targeted questions

State mutation:
Candidate only

Approval:
Human approval before canonicalization

AI-WF-02 — Research

Trigger:
User requests project research

Context:
Research question + approved intent + relevant project state

Tools:
Approved search/retrieval tools

Flow:
Research planning
↓
Search
↓
Extraction
↓
Evidence validation
↓
Finding
↓
Human review

Output:
Evidence records + research findings

Guardrail:
No source → no verified claim

Failure:
Explicit insufficient-evidence state

AI-WF-03 — Story Development

Trigger:
User requests story development

Context:
Story Brain
Research
Characters
Canon
Decisions

Output:
Candidate story artifacts

Validation:
Schema
Canon consistency
Context grounding

Human:
Edit / reject / regenerate / approve

Mutation:
Approved candidate → new project version

AI-WF-04 — Writing

Trigger:
User selects artifact/section

Context:
Relevant approved story
Relevant characters
Relevant canon
Relevant research

Actions:
Draft
Rewrite
Expand
Shorten
Refine
Regenerate

Output:
Candidate artifact version

Guardrail:
Never silently overwrite approved content

AI-WF-05 — Screenplay

Precondition:

Approved Story Version exists

Then:

Approved Story
↓
Context Resolver
↓
Screenplay Agent
↓
Schema Validation
↓
Candidate Screenplay
↓
Human Review

The source architecture explicitly requires approval gating for downstream screenplay progression.

20. RAG / Knowledge Engineering

Ingestion

```
Upload
↓
File validation
↓
Malware/content safety checks
↓
Object storage
↓
Parser
↓
Normalized document
↓
Chunking
↓
Metadata
↓
Embedding
↓
Index
```

Metadata

Every chunk should retain:

```
project_id
input_id
document_id
version_id
section
source_type
authority
created_at
```

Retrieval

```
User intent
↓
Capability
↓
Project authorization
↓
Structured filters
↓
Semantic retrieval
↓
Optional reranking
```

↓
Context package

Important

Vector similarity is **not** authorization.

21. Observability

Required telemetry

Application

```
request_id  
project_id  
actor  
endpoint  
status  
error_code  
latency
```

AI

```
run_id  
agent  
agent_version  
model  
model_version  
prompt_version  
status  
duration  
tokens  
cost  
tool calls  
validation
```

Retrieval

```
query  
source count
```

```
retrieval latency
relevance feedback
```

Product

```
time-to-package
research turnaround
development iterations
approval rate
rejection rate
regeneration rate
project activation
returning projects
```

The engineering blueprint explicitly defines AI run traces and dashboards covering latency, model, prompt version, cost, retrieval quality, continuity and workflow health.

22. Critical AI Trace

Every AI run should be traceable:

```
User
↓
Project
↓
Workflow
↓
Agent + version
↓
Model + provider
↓
Retrieved entity IDs
↓
Tool calls
↓
Output
↓
Validation
↓
Candidate artifact/version
↓
Approval/rejection
↓
Downstream impact
```

Do **not** retain or expose hidden chain-of-thought.

Store observable execution metadata instead.

23. Testing Strategy

23.1 Unit tests

Test:

- domain state transitions
 - approval rules
 - authorization
 - project isolation
 - version creation
 - evidence validation
 - template rules
 - provider adapters
 - package assembly
-

23.2 Integration tests

```
API
↓
Workflow
↓
Project Intelligence
↓
Agent
↓
Model Adapter
↓
Validation
↓
Candidate
```

23.3 E2E

Primary journey:

```
Create Project
→ Intent
→ Story Brain
→ Research
→ Character
→ Story
→ Approval
→ Writing
→ Screenplay
→ Continuity
→ Evaluation
→ Package
```

23.4 Security tests

Mandatory scenarios:

- cross-project retrieval
- unauthorized project mutation
- role escalation
- approval bypass
- prompt injection
- indirect document injection
- malicious web result
- unauthorized export
- secret exposure
- external data leakage

24. AI Evaluation Framework

Evaluation must exist before production launch.

Research

Measure:

- source correctness
- citation completeness
- unsupported claim rate
- source relevance
- provenance correctness

Grounding

Measure:

- project grounding
- context omission
- contradiction with approved state

Character

Measure:

- attribute consistency
- motivation consistency
- relationship consistency
- voice constraints

Continuity

Measure:

Precision
Recall
False-positive rate

The source specifically calls for contradiction precision/recall and false-positive measurement.

Story

Measure:

- human acceptance
- edit distance
- regeneration rate
- rubric evaluation

Product

Primary:

| Time to Human-Approved Story Development Package

Supporting:

- time to useful artifact
- research turnaround
- development iterations
- approved milestone rate
- adoption
- user-reported time saved

25. Non-Functional Requirements

ID	Requirement	Status
NFR-001	Persistent project state	Source
NFR-002	Project isolation	Source
NFR-003	Auditability	Source/Derived
NFR-004	Version history	Source
NFR-005	Secure storage	Source
NFR-006	Relevant context retrieval	Derived
NFR-007	Async long-running AI jobs	Derived

ID	Requirement	Status
NFR-008	Idempotent AI mutations	Derived
NFR-009	AI run observability	Derived
NFR-010	Multiple projects/users support	Source
NFR-011	Graceful AI failure	Derived
NFR-012	Backup/recovery	Required before production
NFR-013	Numeric availability target	TBD
NFR-014	Numeric latency target	TBD
NFR-015	Numeric concurrency target	TBD
NFR-016	RTO/RPO	TBD
NFR-017	AI cost ceiling	TBD

The BRD requires persistent accessibility, security, traceability and support for multiple projects/users/production teams, but does not establish numeric SLOs.

26. Reliability & Failure Handling

Principle

Fail visibly; never silently corrupt project state.

Provider timeout

```

Provider timeout
  ↓
Retryable failure
  ↓
Retry with backoff
  ↓
If exhausted
  ↓
FAILED_RETRYABLE / FAILED_FINAL

```

Invalid model output

```

Model output
  ↓

```

```
Schema validation
↓
INVALID
↓
Reject mutation
↓
Repair/retry
```

Research failure

```
Search unavailable
↓
Preserve project
↓
Show incomplete research state
↓
Allow retry
```

Concurrent version conflict

```
Version A loaded
Version B committed
User attempts mutation against A
↓
Conflict
↓
Reject stale mutation
↓
User reconciles
```

27. Infrastructure & DevOps

Environments

```
Development
↓
Staging
↓
Pilot
↓
Production
```

Development

- isolated credentials
- test/synthetic content preferred

Staging

- production-like architecture
- non-production data

Pilot

- controlled real projects/users
- elevated monitoring

Production

- security hardening
- backups
- monitoring
- recovery controls
- approved provider policies

The engineering specification explicitly requires environment separation and states that production must not depend on developer credentials or shared staging data.

28. CI/CD

Recommended pipeline:



AI model/prompt changes should not bypass the regression suite.

29. Backup & Disaster Recovery

Required:

- database backups
- point-in-time recovery where supported
- object-storage versioning
- lifecycle controls
- restore validation

TBD:

- RPO
- RTO
- retention
- restore frequency
- multi-region requirements

The source engineering specification explicitly leaves RPO/RTO as decisions to be established before production.

30. Key Technical Decisions & Trade-offs

Decision 1 — Modular application vs microservices

Recommendation: Modular application + workers.

Why

V1 needs:

- fast iteration
- strong shared domain model
- centralized policy
- low operational complexity

Microservices should only be introduced when scale, isolation or ownership justifies them.

Decision 2 — Relational DB vs graph DB

Recommendation: Relational database first.

Why:

- project state is highly transactional
- versioning requires transactional semantics
- approvals require consistency
- relationships can initially be represented relationally
- dependency graph can be derived

A dedicated graph database is **not justified by current source requirements**.

Decision 3 — Vector DB

Recommendation: Do not introduce a separate vector infrastructure unless retrieval testing proves it necessary.

A vector capability is useful, but the authoritative Story Brain must remain structured.

Decision 4 — Agent microservices

Recommendation: No.

Logical agent boundaries should exist, but they should initially run inside the orchestration/worker architecture.

Decision 5 — Model provider

Decision: TBD.

The architecture must support provider abstraction.

31. Engineering Dependencies

Blocking dependencies

Dependency	Owner	Why
Story Brain schema	Product + Engineering	Foundation
Version semantics	Product	Required for state
Approval state machine	Product + Engineering	Required for gates
Roles/permissions	Product	Required for authorization
Research Trust Policy	Product + KM	Required for research
Evaluation rubric	Product + AI Storyteller + KM	Required for AI evaluation
Model provider	AI Architecture + Leadership	Required for model gateway
Search providers	AI Architecture + KM	Required for research
Provider data policy	Leadership/Security	IP protection
Data residency	Leadership/Security	Production deployment
Input formats	Product + AI Engineering	Parser scope
Export format	Product	Package implementation
Performance SLOs	Product + Architecture	Production readiness
Cost ceiling	Leadership + AI Architecture	Model routing
RTO/RPO	Leadership + Platform	Production readiness

The engineering source material explicitly identifies these as implementation blockers/open decisions.

32. Major Risks

Risk	Technical response
Incorrect Story Brain memory	Versioned canonical state + validation
Hallucinated research	Evidence/provenance policy
Context pollution	Scoped Context Resolver
Continuity failure	Canon + dependency model
AI quality	Evaluation harness
Creative rejection	Human approval + editability
Cross-project leakage	Server-side isolation
Prompt injection	Untrusted-content boundary
Provider lock-in	Model Gateway

Risk	Technical response
Cost explosion	Token/cost telemetry + routing
Latency	Async workers + scoped context
Version conflicts	Optimistic concurrency
Scope explosion	Golden Workflow + staged implementation
Schema lock-in	Explicit versioned domain model
Downstream staleness	Dependency tracking

The Strategy specifically identifies AI quality, hallucinated research, creative resistance, noisy context, continuity failure, model commoditization and security/IP risk as strategic risks.

33. Recommended Implementation Phases

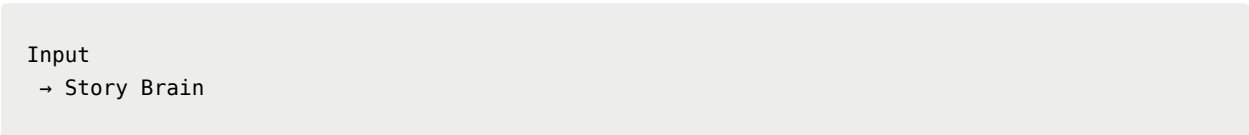
Phase 0 — Architecture Foundation

Build:

- Project Intelligence schema
- Project isolation
- Authentication/authorization foundation
- Version semantics
- Approval state machine
- Model Gateway
- Agent contract
- AI telemetry
- Evaluation harness skeleton

Exit condition

A representative project can move through:



without hidden AI state.

Phase 1 — Core Creative MVP

Build:

- Input
- Story Brain
- Research
- Evidence/provenance
- Character intelligence
- Narrative reasoning
- Story development
- Human approval
- Writing
- Screenplay
- Dialogue
- Canon/continuity
- Evaluation
- Package export
- Version history

This aligns with the MVP direction while keeping Story Doctor and advanced dependency intelligence lighter/later. The Discovery artifact's current hypothesis is explicitly centered on Project Intelligence + Research + Story Development + Canon/Continuity + Human Approval.

Phase 2 — Production Hardening

Build:

- security review

- threat-model closure
- load testing
- resilience testing
- provider failure drills
- AI regression suite
- cost budgets
- operational dashboards
- backup/restore validation
- RTO/RPO validation

Phase 3 — AI Expansion

Potential future capabilities:

- advanced dependency propagation
- deeper autonomy
- advanced Story Doctor
- world intelligence
- collaboration/branching
- visual intelligence
- production workflows

34. Traceability Matrix

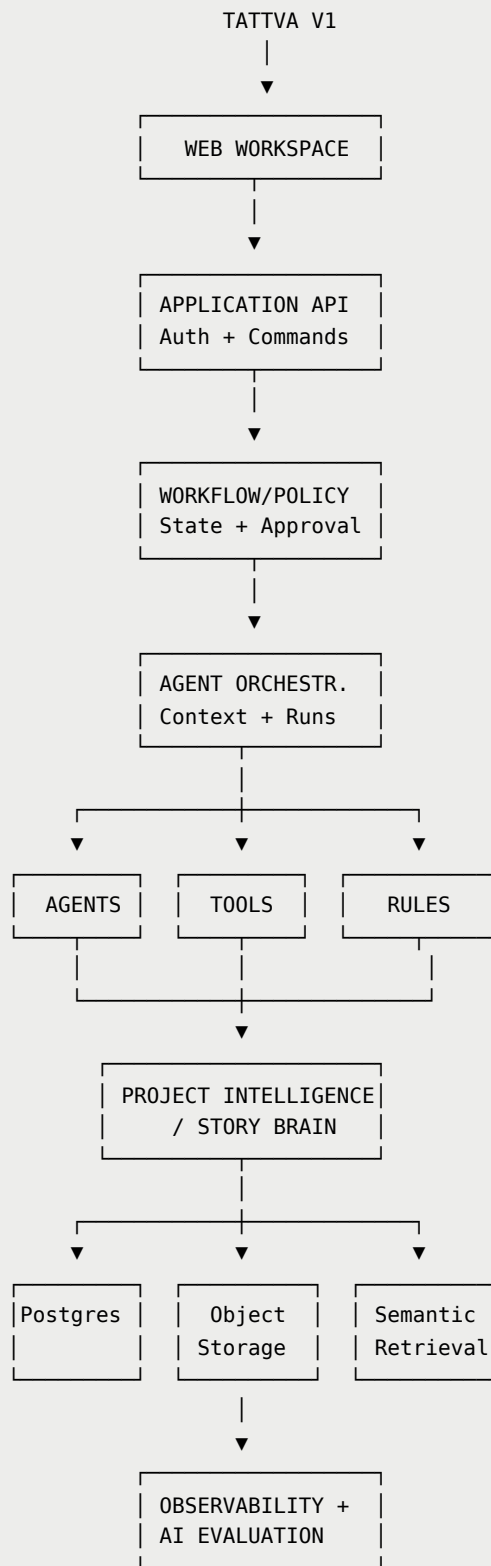
Product requirement	Architecture component	Data	AI	Engineering
PR-001 Create project	Project API	Project/Input	Intent	Project module
PR-002 Story Brain	Project Intelligence	Core entities	Context Resolver	PI Service

Product requirement	Architecture component	Data	AI	Engineering
PR-003 Missing context	Intent	Intent	Intent Agent	Workflow
PR-004 Research provenance	Research Service	Evidence	Research Agent	Worker
PR-005 Character intelligence	PI Service	Character/Relationship	Character Agent	Domain module
PR-006 Narrative reasoning	AI Layer	Findings	Narrative Agent	Agent runtime
PR-007 Story alternatives	Story module	Story/Version	Story Agent	Candidate workflow
PR-008 Context-aware writing	Context Resolver	Approved state	Writing Agent	Writer workflow
PR-009 Canon/continuity	Rules + dependency	Canon/Dependency	Continuity Agent	Deterministic engine
PR-010 Evaluation	Evaluation Service	Evaluation	Evaluation Agent	Eval harness
PR-011 Versioning	Version Manager	ArtifactVersion	All agents	Transaction layer
PR-012 Final package	Export Service	Artifact refs	Optional evaluation	Export worker
PR-013 Security	AuthZ/Policy	Organization/Project	Context restrictions	Security layer
PR-014 AI observability	Observability	AI Run	All workflows	Telemetry

The Unified Specification's requirements explicitly cover project creation, Story Brain, research, character intelligence, narrative reasoning, story generation, writing, canon, evaluation, versioning, package assembly, isolation and AI observability.

35. Final Architecture Decision

The recommended V1 architecture can be summarized as:



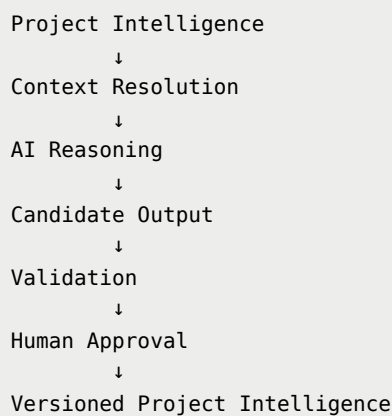
The most important engineering principle

The Story Brain is the system of record.

Not:

- the prompt
- the conversation
- the agent's memory
- the vector database
- the LLM
- the frontend

Everything important eventually flows through:



That architecture directly supports Tattava's stated AI-native principles: persistent project intelligence, project-grounded reasoning, traceability, model agnosticism, validation and human authority.

36. Engineering Readiness Gate

Engineering should **not lock the physical stack or begin full-scale implementation** until these decisions are closed:

Product

- ☐ V1/P0/P1 scope approved

- ☐ Approval workflow finalized
- ☐ Version semantics finalized
- ☐ Genre/format/template scope finalized
- ☐ Input formats finalized
- ☐ Export/package format finalized

AI

- ☐ Model provider(s) selected
- ☐ Model data-retention policy approved
- ☐ Research Trust Policy approved
- ☐ Evaluation rubric approved
- ☐ Golden datasets defined
- ☐ AI quality thresholds defined

Security

- ☐ Identity provider selected
- ☐ RBAC finalized
- ☐ Project isolation model approved
- ☐ Data residency decision made
- ☐ Provider training/retention policy approved
- ☐ Threat model reviewed

Engineering

- ☐ Database selected
- ☐ Queue/job infrastructure selected
- ☐ Object storage selected
- ☐ Retrieval approach selected
- ☐ API standard selected

- ☐ Observability stack selected
- ☐ CI/CD standard selected
- ☐ Backup strategy approved
- ☐ RTO/RPO defined
- ☐ Performance/SLO targets defined
- ☐ AI cost ceiling defined

The Discovery artifact reinforces that the exact P0 feature set and several workflow assumptions still require validation rather than being treated as proven facts.

Bottom Line

The technology should be designed around **one persistent Project Intelligence system with governed AI orchestration**, rather than around fourteen isolated AI agents.

The V1 technical backbone is:

Project + Story Brain → Context Resolver → Workflow/Policy Engine → Agent Orchestrator → Model/Tool Gateway → Candidate → Validation → Human Approval → Versioned Canonical State → Dependency/Continuity → Evaluation → Package.

That gives Engineering a stable foundation for V1 while preserving the architecture needed for the later **Visual Intelligence, AI Filmmaking, Localization, DVS Production Suite and broader Entertainment Operating System** roadmap without prematurely building those systems into V1.